

Budget Control

Product, UI/UX and feature audit

Analysis-only report. No files, migrations, database records, or application behavior were changed as part of this audit.

Executive summary

The product is functionally capable, but its interface presents too many equally weighted features in one narrow, card-heavy shell. The recommended direction is not feature removal by default: it is a clearer product hierarchy built around daily money status, activity, plans, wallet, and settings.

The largest visible gains will come from reducing seven top-level destinations to five, moving forms and advanced tools out of the primary flow, and introducing a responsive application shell.

Keep	Change	Move / merge
Income/expense model, alerts, card limit progress, filters, backup and CSV capability	Navigation, dashboard hierarchy, action density, form behavior, responsive layout	Subscriptions + recurring under Plan; categories + data tools under Settings

1. Visual and UI audit

- The current shell is capped at 480px. It reads as a mobile mockup on desktop and does not use the available screen space.
- Seven text-heavy top navigation tabs have equal visual weight. Primary tasks and secondary setup actions compete for attention.
- The Transactions screen combines four jobs: entry, recurring setup, CSV tools, and history. This is the densest and least focused screen.
- Forms are always shown before lists, so reviewing existing information requires scrolling past setup controls.
- Repeated bordered cards, emoji icons, small labels, and many visible actions make the interface feel busy rather than deliberately layered.
- Subscription, recurring, card, category, and budget rows expose too many text buttons at once. This creates wrapping and inconsistent density on narrow screens.
- Empty and loading states exist, but are plain status text. They do not explain the value of the feature or guide the next action.
- Native confirmation prompts interrupt the visual language and make destructive actions feel less polished.

What already works

- The dashboard includes useful monthly summary, alert, category, and trend information.
- The current light/dark theme foundation, toast feedback, transaction search, pagination, and budget/card progress indicators should be retained.
- The backend API already supports most data needed for a better UI; the highest-impact work is primarily visible frontend restructuring.

2. Duplicate and redundant feature audit

Feature	Overlap	Recommendation	Effect
Recurring toggle and recurring list	Both configure fixed, repeating money movement inside Transactions	Merge into Plan > Scheduled items	Keep capability; remove duplicated mental model
Subscriptions and recurring expenses	Both represent regular future commitments	Keep separate record types, show together under Plan	Remove standalone top-level subscription destination
Categories and budgets	Categories are a budget configuration input	Move Categories to Settings; keep Budgets in Plan	Fewer top-level choices
CSV controls in Transactions	Advanced data operation competes with daily ledger workflow	Move to Settings > Data management	Cleaner transaction screen
Backup in Account	Technical data action mixed with profile	Move into the same Data management area	Cleaner account settings
Dashboard alerts	Links to data shown elsewhere	Retain and condense	Useful priority center, not redundant
Category chart and ledger	Summary versus detail	Retain both; make chart compact	Different user questions remain supported

No core data feature should be deleted in the first redesign phase. The recommended removals are from the primary navigation and from cluttered screens, not from the data model or API.

3. First-time user experience

- A new user is not clearly told where to begin. The empty dashboard should lead with one action: Add your first transaction.
- The transaction form asks for type, frequency, amount, date, category, credit card, and note before the user has built confidence. Default to a simple transaction and disclose advanced fields on demand.
- Editing a record scrolls the user to a distant form, which loses list context. Use a modal or mobile bottom sheet instead.
- The term Bank does not accurately describe a credit-card-only area. Rename it Wallet or Cards.
- The user should see net position, budget health, upcoming payment pressure, and recent activity before charts or configuration details.

Proposed user journey

- Dashboard empty state -> Add first transaction.
- Quick entry -> expense/income, amount, category; date defaults to today.
- Advanced fields -> card, note, future date, or recurring schedule.
- Plan -> configure budgets and regular commitments once the user has activity.

4. Proposed UI structure

Current top level	Proposed top level	Purpose
General overview	Dashboard	Monthly position, alerts, recent activity, concise insights
Transactions	Activity	Searchable ledger and focused new transaction action
Subscriptions + Budget	Plan	Budgets, subscriptions, and scheduled transactions
Bank	Wallet	Credit-card limits, debt, and due dates
Categories + Account	Settings	Categories, profile, security, CSV, backup/restore

Responsive behavior: on mobile, use a fixed bottom navigation for the five primary destinations and a prominent add button. At desktop width, use a compact sidebar and a two-column content layout where appropriate.

5. Prioritized implementation plan

Change	Problem	Visible result	Risk	Priority	Merge/remove
Responsive app shell	Fixed narrow layout	Professional mobile and desktop layouts	Medium	High	None
Five-area navigation	Seven equal tabs	Clearer hierarchy	Medium	High	Merge top-level screens
Dashboard redesign	Flat card stack	Net-status-led overview and action cues	Low-Medium	High	Regroup charts
Quick add sheet	Always-open complex form	Fast, guided entry	Medium	High	Separate advanced fields
Activity ledger	Transaction page is overloaded	Filter-first readable history	Medium	High	Move CSV tools
Overflow row actions	Too many visible buttons	Calmer lists and consistent actions	Low	High	Merge secondary actions
Plan workspace	Regular money features scattered	Budgets and commitments viewed together	Medium	Medium	Merge subscriptions/recurring UI
Settings and data tools	Technical tools interrupt daily workflow	Cleaner everyday screens	Low	Medium	Move category/CSV/backup
State and dialog system	Plain states and native confirms	Polished empty, loading, error and confirmation moments	Medium	Medium	Unify confirms
Visual design system	Inconsistent emoji/button/spacing language	Typography, icon, color and spacing consistency	Low	High	Unify styles

6. Detailed change checklist

Responsive shell

Current problem: the application is permanently constrained to a 480px center column. Proposed solution: responsive sidebar or bottom navigation with adaptive content width. Visible result: a product that feels designed for both desktop and mobile. Likely area: static UI shell. Risk: medium. Priority: high. Merge/remove: none.

Dashboard hierarchy

Current problem: all dashboard cards compete equally. Proposed solution: prominent net-status summary, secondary metric chips, focused alerts, recent activity, then compact charts. Visible result: the user understands their situation at a glance. Likely areas: dashboard UI and potentially one lightweight recent-activity API query. Risk: low-medium. Priority: high. Merge/remove: regroup existing cards.

Focused transaction entry

Current problem: the primary form is always open and exposes advanced decisions immediately. Proposed solution: quick-add sheet with progressive disclosure. Visible result: less intimidating, faster entry. Likely area: transactions UI. Risk: medium. Priority: high. Merge/remove: scheduled entry moves out of the main form.

Plan workspace

Current problem: budgets, subscriptions, and scheduled items live in separate destinations. Proposed solution: one Plan area with clear subviews. Visible result: recurring commitments and spending limits feel connected. Likely area: static UI. Risk: medium. Priority: medium. Merge/remove: subscriptions and recurring UI merge at navigation level.

Consistent state patterns

Current problem: generic text empty states, mixed button treatments, and browser confirms. Proposed solution: reusable empty panels, skeletons, in-app dialogs, icon buttons, and overflow menus. Visible result: more coherent, intentional feedback. Likely area: static UI. Risk: medium. Priority: medium. Merge/remove: all confirmation paths unify.

Recommended first delivery: implement the High priority items as one visual product pass while preserving the existing API and data behavior. Then review the result before deciding whether Medium priority feature grouping needs any backend support.